

UNIVERSIDADE DE PERNAMBUCO
ESCOLA POLITÉCNICA DE PERNAMBUCO

Maria Eduarda Neves Oliveira Bezerra

Mateus Freitas Gomes

Lucius' Gabriel Venceslau de Vasconcelos

ANALISANDO O LOG

Relatório Técnico – Programação Orientada a Objetos

Caruaru

2026

Maria Eduarda Neves Oliveira Bezerra

Mateus Freitas Gomes

Lucius' Gabriel Venceslau de Vasconcelos

ANALISANDO O LOG

Relatório técnico apresentado à disciplina de
Programação Orientada a Objetos da
Universidade de Pernambuco, como requisito
parcial de avaliação.

Professor(a): Jackson Raniel Florêncio da Silva

Caruaru

2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 DESCRIÇÃO DO PROBLEMA.....	4
3 DIAGRAMA DE CLASSES.....	5
4 DECISÕES DE PROJETO E CÓDIGO FONTE.....	5
4.1 LogEntry.....	5
4.2 LogParser.....	6
4.3 LogEntryBuffer.....	7
4.4 LogThread.....	7
4.5 LogReaderManager.....	8
4.6 LogAnalyzer.....	8
4.7 WriteFile.....	9
4.8 Menu.....	10
4.9 Main.....	10
5 PROGRAMAÇÃO CONCORRENTE.....	11
6 COMPATIBILIDADE ENTRE SISTEMAS OPERACIONAIS.....	11
7 VERSIONAMENTO COM GIT.....	12
8 CONCLUSÃO.....	12

1 INTRODUÇÃO

Este relatório descreve o desenvolvimento de um programa em Java capaz de analisar arquivos de log de servidores Web Apache (access.log). O sistema foi construído com base nos princípios da Programação Orientada a Objetos (POO), utilizando conceitos como encapsulamento, responsabilidade única, imutabilidade e programação concorrente com threads.

O arquivo access.log contém registros de requisições HTTP realizadas por clientes a um servidor, incluindo informações como endereço IP, data e hora, tipo de requisição, recurso solicitado, código de resposta HTTP, tamanho do objeto devolvido, endereço de referência e informações do user agent.

O programa oferece um menu interativo com quatro opções de análise: filtragem de recursos grandes respondidos com sucesso, listagem de requisições não respondidas em novembro de 2021, percentual de acessos por sistema operacional em 2021 e cálculo da média de tamanho das requisições POST com sucesso em 2021.

O repositório público do projeto está disponível em:
<https://github.com/usermenob/AnalisandoLOG.git>

2 DESCRIÇÃO DO PROBLEMA

O objetivo do trabalho é processar um arquivo de log com um milhão de registros de forma eficiente, extraindo informações relevantes e apresentando resultados por meio de arquivos de saída. As principais dificuldades envolvem o processamento de grandes volumes de dados, a correta interpretação do formato do arquivo de log por meio de expressões regulares (regex) e a garantia de compatibilidade entre diferentes sistemas operacionais.

Para otimizar o desempenho no processamento do arquivo, optou-se por dividir a leitura em múltiplas threads paralelas, cada uma responsável por um bloco de bytes do arquivo. Após a leitura, as entradas são armazenadas em uma estrutura thread-safe e disponibilizadas para análise.

3 DIAGRAMA DE CLASSES

O diagrama de classes a seguir representa a estrutura do sistema, evidenciando as responsabilidades de cada classe e os relacionamentos entre elas. O sistema é composto por nove classes distribuídas entre as camadas de leitura/parsing (desenvolvidas pela Pessoa 1) e análise/saída (desenvolvidas pela Pessoa 2).

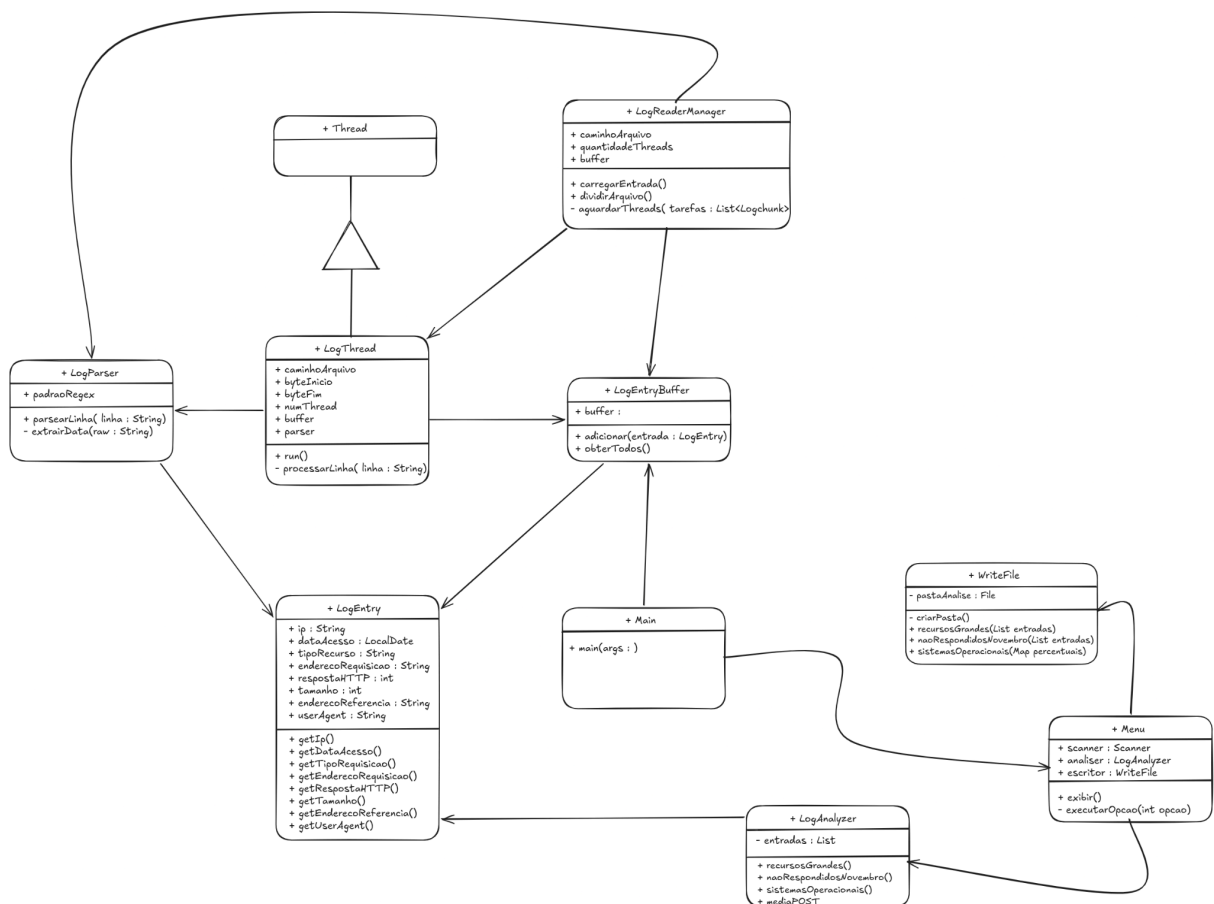


Figura 1 – Diagrama de Classes do Sistema

4 DECISÕES DE PROJETO E CÓDIGO FONTE

4.1 LogEntry

A classe LogEntry representa uma única linha do arquivo de log como um objeto Java. Todos os seus atributos são declarados como `private final`, tornando os objetos imutáveis após a criação. Essa decisão fundamenta-se no princípio do encapsulamento da POO: ao impedir que qualquer classe externa altere os dados de uma entrada já processada, evitam-se bugs em ambiente multithread, onde diversas threads poderiam tentar modificar o mesmo objeto simultaneamente.

A imutabilidade é uma técnica amplamente recomendada em programação concorrente, pois objetos imutáveis são intrinsecamente thread-safe, eliminando a necessidade de sincronização adicional para leitura. A classe não possui setters, apenas getters, o que reforça esse princípio.

O atributo `dataAcesso` utiliza o tipo `LocalDate` da API `java.time`, introduzida no Java 8. Essa escolha facilita as operações de filtragem por mês e ano exigidas pelas opções 2, 3 e 4 do menu, por meio dos métodos `getMonthValue()` e `getYear()`.

```
public class LogEntry {
    private final String ip;
    private final LocalDate dataAcesso;
    private final String tipoRequisicao;
    private final String enderecoRecurso;
    private final int respostaHTTP;
    private final int tamanho;
    private final String enderecoReferencia;
    private final String userAgent;
    // construtor e getters...
}
```

4.2 LogParser

A classe LogParser é responsável por converter uma linha de texto bruto do arquivo de log em um objeto LogEntry. Ela encapsula a expressão regular (regex) compilada como atributo, evitando que o padrão seja recompilado a cada chamada — o que seria custoso em termos de desempenho com um milhão de registros.

O método `parsearLinha()` aplica o padrão ao texto recebido e extrai cada campo usando grupos de captura do Matcher. Caso a linha não corresponda ao formato esperado, o método retorna null, garantindo robustez contra linhas malformadas no arquivo.

O método privado `extrairData()` converte a string de data do formato dd/MMM/yyyy (ex: 25/May/2022) para um objeto `LocalDate` usando `DateTimeFormatter` com `Locale.ENGLISH`, necessário para interpretar corretamente os nomes dos meses em inglês presentes no arquivo de log.

4.3 LogEntryBuffer

O LogEntryBuffer atua como repositório compartilhado entre as threads de leitura. Seu atributo `buffer` é uma `ConcurrentLinkedQueue`, estrutura da API `java.util.concurrent` que garante operações thread-safe sem necessidade de bloqueios explícitos (locks), oferecendo alto desempenho em cenários de múltiplos produtores.

O método `adicionar()` é chamado por cada thread para inserir uma entrada processada no buffer. O método `obterTodos()` é chamado apenas após todas as threads encerrarem sua execução, eliminando risco de condição de corrida no momento da leitura da lista completa.

4.4 LogThread

A classe `LogThread` estende `Thread` e representa a unidade de execução paralela do sistema. Cada instância é responsável por ler um intervalo específico de bytes do arquivo de log, utilizando `RandomAccessFile` para posicionar o ponteiro de leitura diretamente no byte inicial atribuído.

Um cuidado importante implementado é o descarte da primeira linha quando o byte inicial não é zero. Isso ocorre porque a divisão por bytes pode cortar uma linha ao meio, e a thread anterior já terá lido essa linha até o final. O loop de leitura continua enquanto o ponteiro do arquivo não ultrapassa o byte final atribuído, garantindo que a última linha do bloco seja lida por completo mesmo que ultrapasse levemente o limite.

O método privado `processarLinha()` delega o parsing para o `LogParser` e, se o resultado não for nulo, adiciona a entrada ao buffer compartilhado.

4.5 LogReaderManager

O `LogReaderManager` centraliza a orquestração da leitura paralela. Ele recebe o caminho do arquivo, a quantidade de threads desejada e o buffer compartilhado. O método público `carregarEntrada()` coordena toda a sequência: calcula o tamanho total do arquivo, divide os intervalos de bytes, instancia e inicia as threads, aguarda todas encerrarem via `join()` e retorna a lista completa de entradas.

O método privado `dividirArquivo()` calcula os intervalos de forma que a última thread sempre cubra até o final do arquivo, compensando possíveis imprecisões da divisão inteira. O método privado `aguardarThreads()` itera sobre a lista de threads e chama `join()` em cada uma, garantindo que o programa não avance para a análise antes de todas as leituras estarem completas.

4.6 LogAnalyzer

O LogAnalyzer recebe a lista completa de LogEntry e implementa os quatro métodos de análise exigidos pelo projeto. A classe segue o princípio da responsabilidade única da POO: ela apenas filtra e calcula — a responsabilidade de persistir os resultados em arquivo pertence ao WriteFile.

O método recursosGrandes() filtra entradas com código de resposta entre 200 e 299 e tamanho superior a 2000 bytes, atendendo à opção 1 do menu. O método naoRespondidosNovembro() filtra códigos entre 400 e 499 com data em novembro de 2021, usando getMonthValue() == 11 e getYear() == 2021 do LocalDate.

O método sistemasOperacionais() percorre todas as entradas do ano de 2021 e identifica o sistema operacional pelo campo userAgent usando contains(). A verificação de Android e Mobile é feita antes das demais, conforme determinado pelo enunciado. Ao final, converte as contagens em percentuais dividindo pelo total de acessos do ano e multiplicando por 100.

O método mediaPOST() acumula o tamanho de todas as requisições POST respondidas com sucesso em 2021 usando uma variável long para evitar overflow, e retorna a média como double. Caso não haja registros que atendam aos critérios, retorna zero para evitar divisão por zero.

```
public double mediaPOST() {
    long soma = 0;
    int count = 0;
    for (LogEntry entrada : entradas) {
        if (entrada.getTipoRequisicao().equals("POST")
            && entrada.getRespostaHTTP() >= 200
            && entrada.getRespostaHTTP() <= 299
            && entrada.getDataAcesso().getYear() == 2021) {
            soma += entrada.getTamanho();
            count++;
        }
    }
    return count == 0 ? 0 : (double) soma / count;
}
```

```
        }  
    }  
    return count == 0 ? 0 : (double) soma / count;  
}
```

4.7 WriteFile

O WriteFile encapsula toda a lógica de persistência dos resultados em arquivo. Sua principal responsabilidade é garantir que os arquivos sejam salvos na pasta Análise de forma compatível com qualquer sistema operacional, independentemente do diretório de execução do programa.

O atributo pastaAnalise utiliza `new File(System.getProperty("user.dir"), "Análise")`, onde `user.dir` retorna o diretório atual de execução. O construtor `new File(diretório, nome)` monta o caminho usando o separador correto para cada SO automaticamente, evitando o uso hardcoded de `/` ou `\`.

O método `criarPasta()` chama `mkdirs()`, que cria a pasta apenas se ela não existir, retornando `false` silenciosamente caso já exista. Cada método de escrita utiliza o padrão `try-with-resources` com `BufferedWriter`, que fecha o arquivo automaticamente ao final do bloco, mesmo em caso de exceção, evitando vazamento de recursos.

```
private final File pastaAnalise =  
    new File(System.getProperty("user.dir"), "Análise");  
  
private void criarPasta() {  
    pastaAnalise.mkdirs();  
}
```

4.8 Menu

A classe Menu é responsável pela interface com o usuário. Ela recebe por injeção de dependência um LogAnalyzer e um WriteFile, seguindo o princípio da inversão de

dependência: o Menu não instancia suas dependências, apenas as utiliza. Isso facilita a manutenção e o teste de cada componente de forma isolada.

O método `exibir()` implementa um laço `do-while` que garante que o menu seja exibido ao menos uma vez antes de verificar a condição de saída. O método privado `executarOpcao()` utiliza `switch/case` para despachar a execução para o par analisador/escritor correspondente a cada opção. A opção 4 imprime o resultado diretamente no terminal, sem gerar arquivo, conforme especificado no enunciado.

4.9 Main

A classe Main é o ponto de entrada do programa. Ela instancia o `LogEntryBuffer`, o `LogReaderManager` e aciona `carregarEntrada()`, que dispara as threads e aguarda sua conclusão retornando a lista de entradas. Em seguida, instancia o `LogAnalyzer` com essa lista, o `WriteFile` e o `Menu`, iniciando o loop interativo.

```
public static void main(String[] args) {
    LogEntryBuffer buffer = new LogEntryBuffer();
    LogReaderManager gerenciador =
        new LogReaderManager("access.log", 4, buffer);
    List<LogEntry> entradas = gerenciador.carregarEntrada();
    LogAnalyzer analisador = new LogAnalyzer(entradas);
    WriteFile escritor = new WriteFile();
    Menu menu = new Menu(analisador, escritor);
    menu.exibir();
}
```

5 PROGRAMAÇÃO CONCORRENTE

A principal motivação para o uso de threads é o desempenho no processamento de um arquivo com um milhão de registros. Ao dividir o arquivo em blocos de bytes e atribuir cada bloco a uma thread separada, o programa explora o paralelismo de hardware disponível nos processadores modernos com múltiplos núcleos.

A segurança no acesso concorrente à estrutura de dados compartilhada é garantida pelo uso de `ConcurrentLinkedQueue` no `LogEntryBuffer`, uma fila não bloqueante e thread-safe da API `java.util.concurrent`. Isso elimina a necessidade de blocos `synchronized`, que poderiam criar gargalos de desempenho ao serializar o acesso das threads.

O método `join()` utilizado no `LogReaderManeger` garante que a thread principal aguarde todas as threads de leitura encerrarem antes de prosseguir para a análise dos dados, evitando que o `LogAnalyzer` opere sobre uma lista incompleta.

6 COMPATIBILIDADE ENTRE SISTEMAS OPERACIONAIS

Uma exigência explícita do projeto é que a criação da pasta Análise e dos arquivos de saída funcione em qualquer sistema operacional. A solução adotada utiliza a classe `java.io.File` com o construtor de dois argumentos `new File(diretório, nome)`, que internamente utiliza o separador de caminhos correto para cada plataforma: `/` no Linux e macOS, e `\` no Windows.

O método `System.getProperty("user.dir")` garante que o caminho base seja sempre o diretório de execução atual do programa, independentemente de onde o binário esteja instalado. O método `makedirs()` trata silenciosamente o caso em que a pasta já existe, evitando erros em execuções consecutivas.

7 VERSIONAMENTO COM GIT

O projeto foi versionado com Git e armazenado em repositório público no GitHub. Cada integrante da equipe configurou seu nome e e-mail globalmente no Git para que as contribuições de cada membro sejam identificáveis no histórico de commits do repositório.

A estratégia de branches adotada foi a divisão em duas branches de funcionalidade: feature/modelos, responsável pelo LogEntry e classes de leitura, e feature/analise, responsável pelo LogAnalyzer, WriteFile e Menu. Após a conclusão e validação de cada branch, o código foi integrado à branch principal main por meio de merge.

O endereço do repositório público é:
<https://github.com/usermenob/AnalisandoLOG.git>.

8 CONCLUSÃO

O projeto demonstrou na prática a aplicação dos princípios da Programação Orientada a Objetos no desenvolvimento de um sistema real de análise de logs. A divisão em classes com responsabilidades bem definidas facilitou o desenvolvimento paralelo entre os integrantes da equipe e resultou em um código mais organizado, legível e manutenível.

O uso de threads para leitura paralela do arquivo evidenciou a importância de estruturas de dados thread-safe e da sincronização correta entre threads produtoras e consumidoras. A compatibilidade multiplataforma foi garantida por meio das APIs modernas do Java para manipulação de arquivos e caminhos.

Como possível melhoria futura, o número de threads poderia ser configurado dinamicamente com base na quantidade de núcleos disponíveis no processador, utilizando `Runtime.getRuntime().availableProcessors()`, otimizando ainda mais o desempenho em diferentes ambientes de execução.